Advanced JavaScript Course for RAG

An advanced JavaScript document designed for Retrieval-Augmented Generation (RAG), covering modern concepts, architecture, performance, async behavior, patterns, and real-world examples.

1. Execution Context & Call Stack

JavaScript runs inside execution contexts.

Types:
- Global Execution Context
- Function Execution Context
- Eval Execution Context

Call Stack:
JavaScript uses a stack to execute functions.

Example:
```
function first() {
    second();
}

function second() {
    console.log("Hello");
}

first();
```

Concepts:
- Stack overflow
- Memory allocation
- Hoisting behavior

2. Scope, Lexical Environment & Closures

Scope determines variable accessibility.

Types:
- Global scope
- Function scope
- Block scope

Closure:
A closure remembers variables from its outer scope.

Example:
```
function counter() {
    let count = 0;

    return function() {
        count++;
        return count;
    };
}
```

```
const increment = counter();
```

Use cases:
- Data privacy
- Factory functions
- State management

3. Hoisting & Temporal Dead Zone

Hoisting moves declarations to memory phase.

var:
- Hoisted with undefined

let/const:
- Hoisted but inaccessible in TDZ

Example:
```
console.log(a);
var a = 5;
```

4. This Keyword Deep Dive

The value of 'this' changes based on execution context.

Examples:
- Global object
- Object method
- Constructor function
- Arrow functions

Example:
```
const user = {
    name: "Mohammed",
    greet() {
        console.log(this.name);
    }
};
```

5. Prototype & Prototype Chain

JavaScript uses prototype-based inheritance.

Example:
```
function Person(name) {
    this.name = name;
}

Person.prototype.sayHello = function() {
    console.log("Hello");
};
```

Prototype chain enables inheritance lookup.

6. Classes & OOP in JavaScript

ES6 classes simplify prototype syntax.

Example:
```
class User {
    constructor(name) {
        this.name = name;
    }

    greet() {
        return this.name;
    }
}
```

Concepts:
- Inheritance
- Encapsulation
- Polymorphism
- Static methods

7. Functional Programming Concepts

Functional programming in JS includes:

- Pure functions
- Immutability
- Higher-order functions
- Composition

Examples:
map()
filter()
reduce()

Avoid side effects.

8. Event Loop, Microtasks & Macrotasks

JavaScript is single-threaded but asynchronous.

Queue types:
- Call Stack
- Callback Queue
- Microtask Queue

Example:
```
console.log(1);

setTimeout(() => console.log(2), 0);

Promise.resolve().then(() => console.log(3));

console.log(4);
```

Expected output:

1
4
3
2

9. Promises Deep Dive

Promise states:
- Pending
- Fulfilled
- Rejected

Methods:
then()
catch()
finally()

Utilities:
Promise.all()
Promise.race()
Promise.allSettled()
Promise.any()

10. Async/Await Patterns

Async/await improves readability.

Example:

```
async function fetchUser() {
    try {
        const response = await fetch("/user");
        const data = await response.json();
        return data;
    } catch (error) {
        console.log(error);
    }
}
```

Patterns:
- Parallel requests
- Sequential requests
- Error boundaries

11. Memory Management & Garbage Collection

JavaScript automatically manages memory.

Problems:
- Memory leaks
- Detached DOM nodes
- Global references

Best practices:
- Remove listeners
- Clear intervals

- Avoid unnecessary references

12. Modules (ESM) & Code Organization

Modules help split applications.

Export:
export const api = "hello";

Import:
import { api } from "./api.js";

Benefits:
- Maintainability
- Reusability
- Scalability

13. Design Patterns in JavaScript

Popular patterns:
- Module Pattern
- Factory Pattern
- Singleton Pattern
- Observer Pattern
- Pub/Sub Pattern

Useful in scalable apps.

14. Error Handling & Custom Errors

Custom error example:

```
class ValidationError extends Error {
    constructor(message) {
        super(message);
        this.name = "ValidationError";
    }
}
```

Use try/catch strategically.

15. Performance Optimization

Optimization techniques:
- Debounce
- Throttle
- Memoization
- Lazy loading
- Virtualization

Avoid:
- Unnecessary re-renders
- Blocking loops

16. DOM Performance & Rendering

Rendering optimization:
- Minimize DOM updates
- Batch DOM changes
- Use DocumentFragment
- Avoid layout thrashing

17. Security in JavaScript

Common vulnerabilities:
- XSS (Cross-site scripting)
- CSRF
- Injection attacks

Best practices:
- Sanitize input
- Escape output
- Avoid unsafe eval()

18. Advanced ES6+ Features

Topics:
- Destructuring
- Rest/Spread
- Generators
- Iterators
- Optional chaining
- Nullish coalescing
- Symbol
- BigInt

19. Testing in JavaScript

Testing concepts:
- Unit testing
- Integration testing
- End-to-end testing

Popular tools:
- Jest
- Vitest
- Cypress

20. Real-world Architecture Concepts

Patterns:
- MVC
- Component-based architecture
- State management
- API abstraction layer

Used in React, Next.js, and scalable systems.